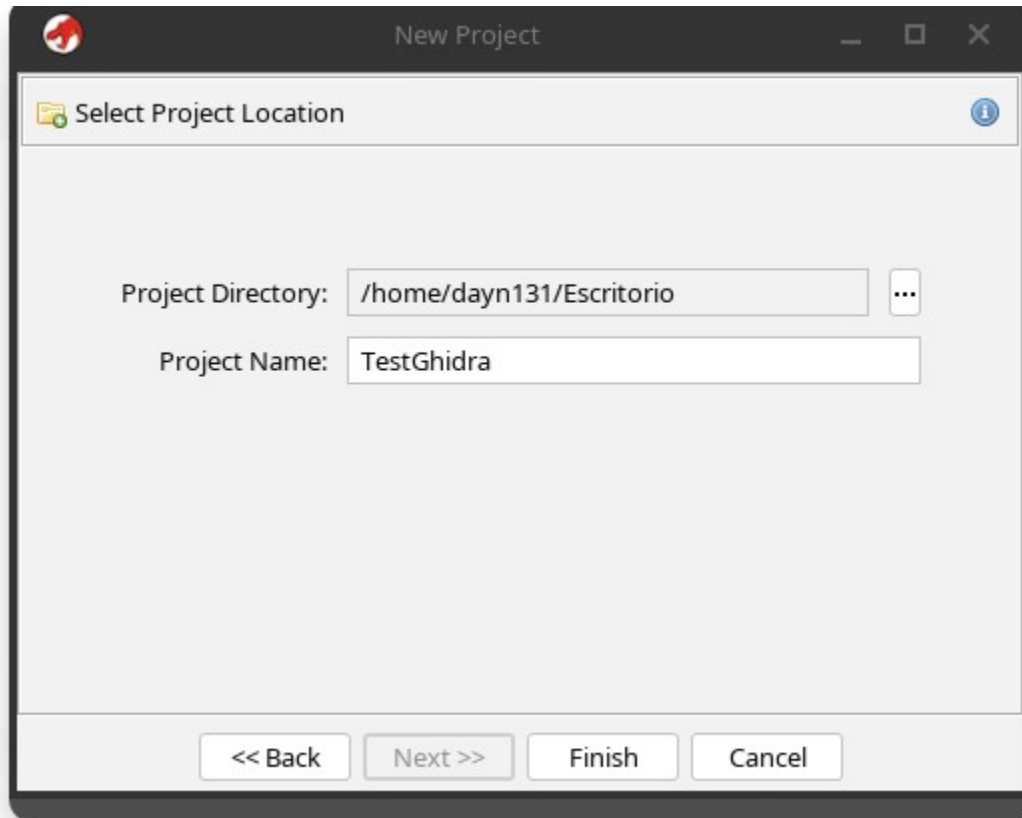
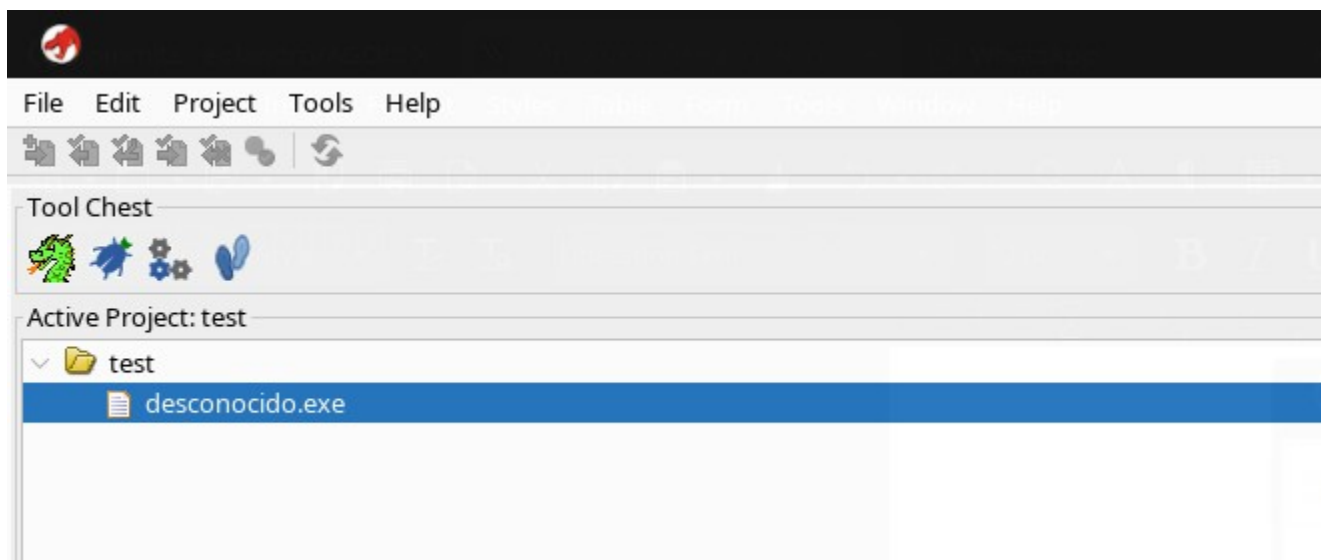


Reconstruccion del Codigo Fuente con Ghidra – Equipo Magenta

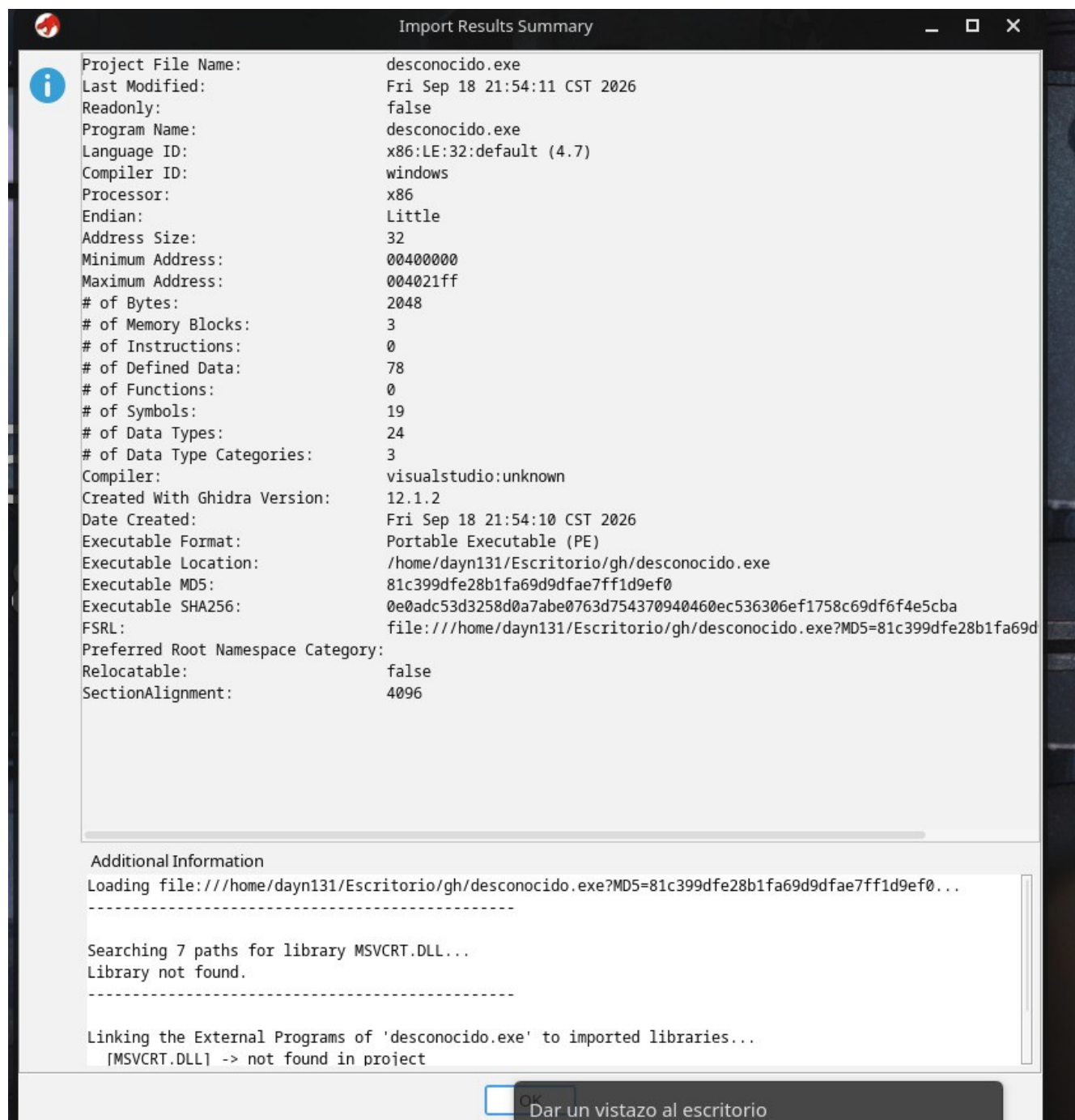
1. Crearemos el proyecto donde analizaremos la muestra.



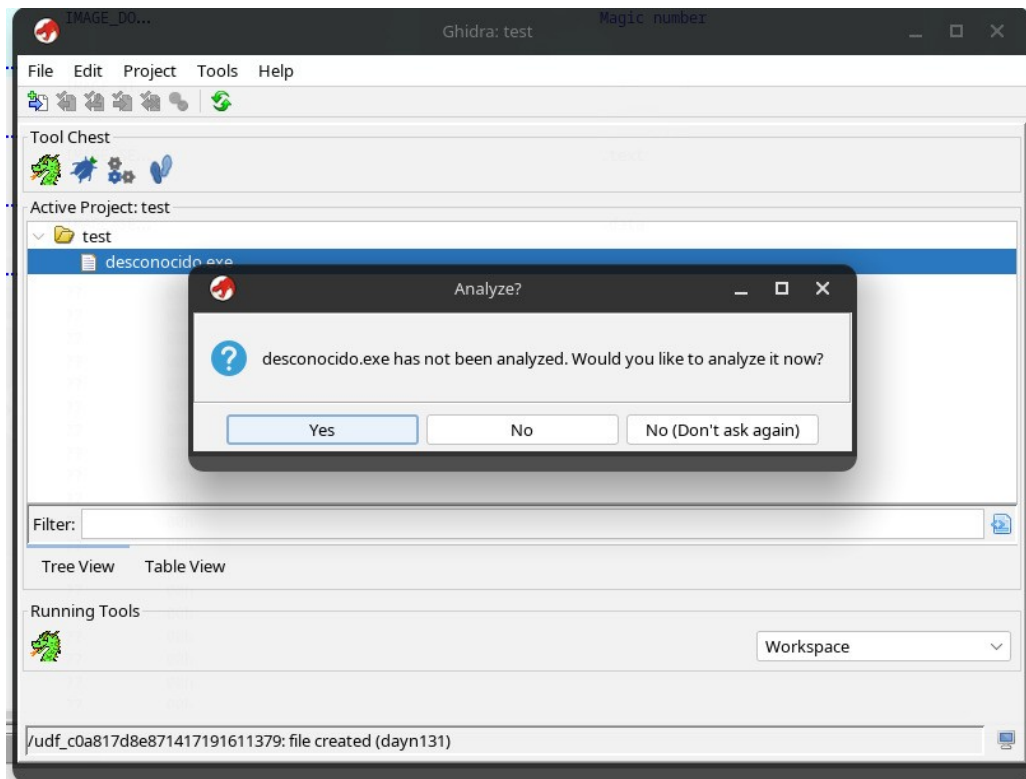
2. Agregaremos la muestra a analizar solo arrastrandola a la herramienta.



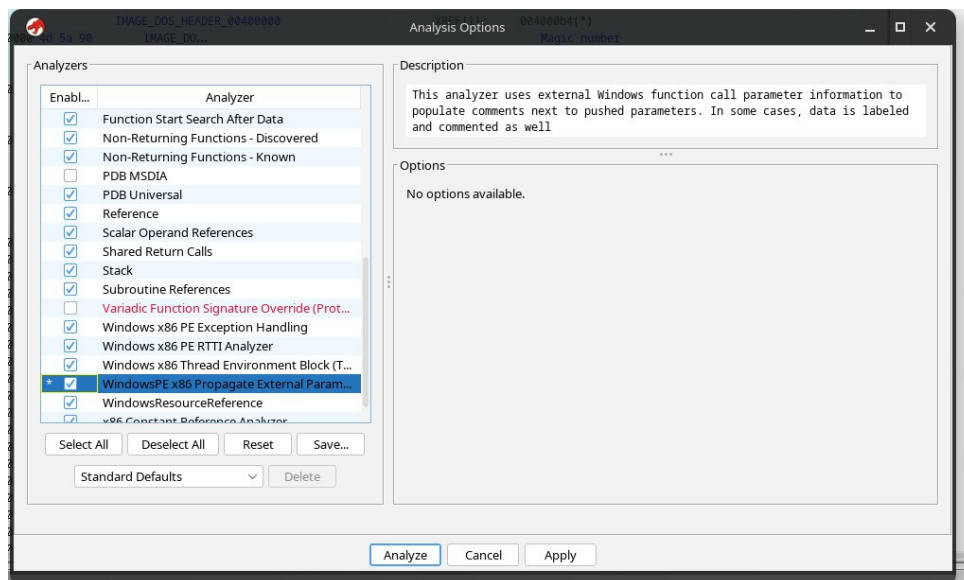
3. Una vez completado nos dara el resumen de la importacion.



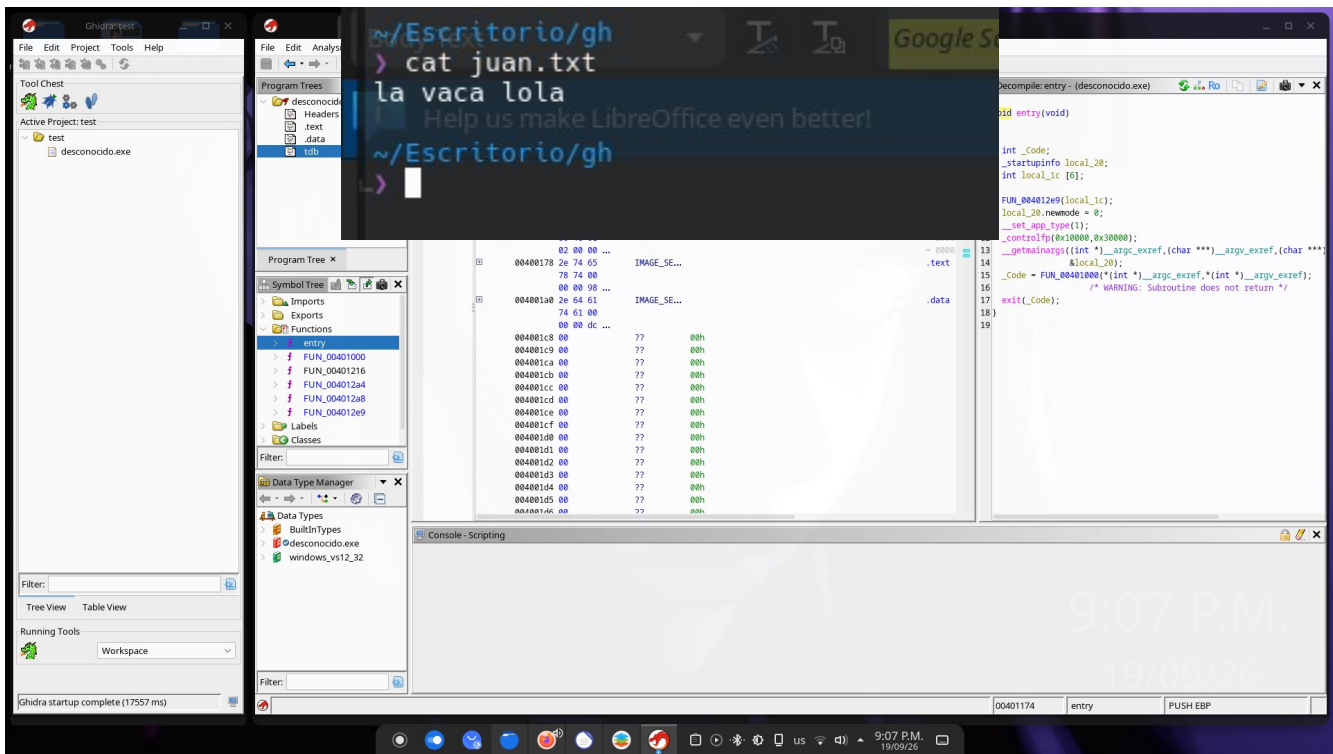
4. Le daremos doble click a la muestra para analizarla



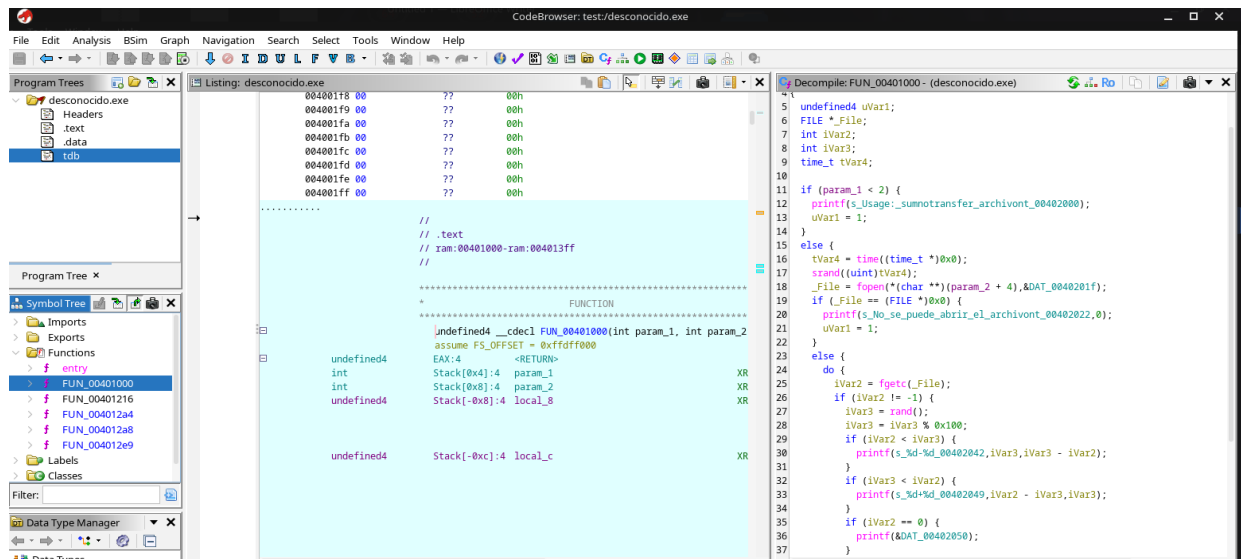
5. Dandole click a la opcion de Propagate External Parameter.



6. Una vez completado nos mostrar el código fuente en ensamblador, con la herramienta de ghidra podemos ver varias cosas el decompilador nos muestra el pseudocódigo en C de secciones que seleccionemos.



7. **Identificación de la función principal:** Dentro de la vista *Symbol Tree*, se examinaron las funciones etiquetadas con la marca Fun. Tras analizar el código en el descompilador, se determinó la función principal tomando en cuenta su volumen de código y la complejidad de sus instrucciones.



8. Analisis del codigo decompilado y reconstruccion.

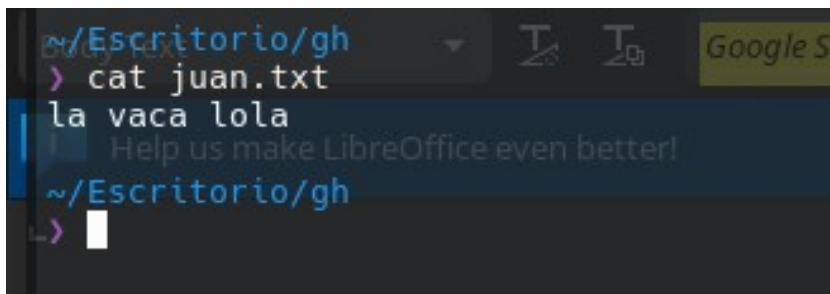
8.1 Variables identificadas:

- **param_1 (argc):** Es el contador de argumentos de la línea de comandos. Sirve para identificar que el usuario paso el numero de parametros necesarios desde la consola en este caso el nombre de un archivo.
- **param_2 (argv):** Es el arreglo con los argumentos de la línea de comandos. `param_2 + 4` (o `argv[1]`) apunta al nombre del archivo que se procesara.
- **uVar1 (estado_salida):** Almacena la respuesta del programa (0 si terminó con éxito, 1 si hubo un error).
- **_File (archivo):** Es el puntero del sistema de archivos hacia el archivo abierto con `fopen`.
- **iVar2 (byte_actual):** Almacena el valor numérico (de 0 a 255) del byte o carácter que el programa está leyendo del archivo en ese instante. Si vale -1, significa que llegó al final del archivo.
- **iVar3 (llave_aleatoria):** Un número aleatorio entre 0 y 255 generado en cada ciclo.
- **tVar4 (tiempo_actual):** Guarda la hora actual en segundos. Se usa como una semilla para `srand()`, asegurando que los números aleatorios cambien cada segundo y no se repita el mismo patrón.

8.2 Analisis y funcionalidad el ejecutable

Básicamente es un programa que oculta el contenido de un archivo usando operaciones matemáticas. Le pasamos un archivo como argumento y el programa va leyendo cada byte uno por uno. Para que nadie pueda ver lo que hay escrito a simple vista, el programa genera números al azar y cambia cada carácter por una suma o una resta que da como resultado ese mismo carácter por ejemplo, en vez de mostrar una A, imprimiria algo como 100-35.

Como prueba de ello corri el ejecutable con el archivo `juan.txt` donde el contenido del archivo es: la vaca lola

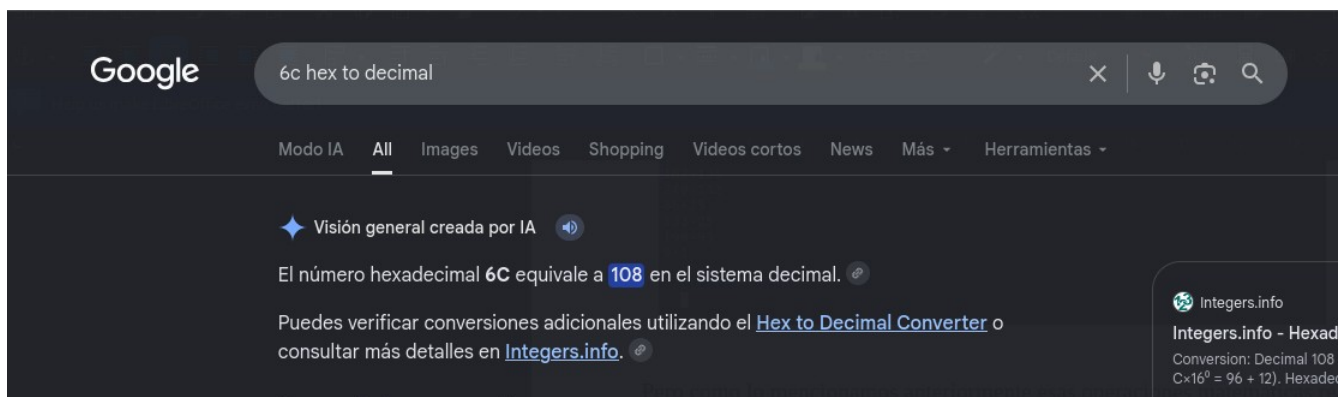
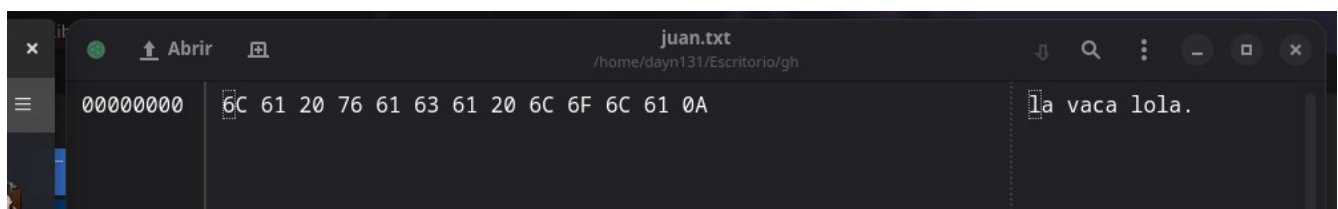


```
~/Escritorio/gh
> cat juan.txt
la vaca lola
Help us make LibreOffice even better!
~/Escritorio/gh
>
```

Y la salida eran las operaciones matematicas de abajo.

```
~/Escritorio
> cd gh
~/Escritorio/gh
> wine desconocido.exe juan.txt
0074:fixme:keyboard:klid_from_xkb_layout Unknown variant "mac"
0074:fixme:keyboard:klid_from_xkb_layout Unknown variant "mac-iso"
MESA-EGL: warning: pci id for fd 33: 10de:2504, driver (null)
pci id for fd 34: 10de:2504, driver (null)
pci id for fd 35: 10de:2504, driver (null)
MESA-EGL: warning: egl: failed to create dri2 screen
MESA-EGL: warning: pci id for fd 33: 10de:2504, driver (null)
pci id for fd 34: 10de:2504, driver (null)
pci id for fd 35: 10de:2504, driver (null)
MESA-EGL: warning: egl: failed to create dri2 screen
MESA-EGL: warning: pci id for fd 33: 10de:2504, driver (null)
0034:fixme:keyboard:klid_from_xkb_layout Unknown variant "mac"
0034:fixme:keyboard:klid_from_xkb_layout Unknown variant "mac-iso"
59+49
191-94
210-178
152-34
183-86
183-84
144-47
167-135
240-132
86+25
133-25
190-93
9+1
~/Escritorio/gh 6s
```

Pero como lo mencionamos anteriormente esas operaciones matematicas realmente nos dan el valor en decimal de que byte es cada cual por ejemplo 59+49 en decimal nos da 108 que en hexadecimal nos da 6C, esto lo corroboramos abriendo el archivo de juan.txt en un editor hexadecimal



Entonces una vez entendiendo el funcionamiento del ejecutable pasamos a reconstruir el código en C y compilarlo con la herramienta de tcc.

```
home ▶ dayn131 ▶ Descargas ▶ tcc ▶ C reconstruido.c
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <time.h>
4
5  int main(int argc, char *argv[]) {
6      int estado_salida;
7      FILE *archivo;
8      int byte_actual;
9      int num_aleatorio;
10     time_t tiempo_actual;
11
12     if (argc < 2) {
13         printf("Usage: sumnotransfer <archivo>\n");
14         estado_salida = 1;
15     }
16     else {
17
18         tiempo_actual = time(NULL);
19         srand((unsigned int)tiempo_actual);
20
21         archivo = fopen(argv[1], "rb");
22
23         if (archivo == NULL) {
24             printf("No se puede abrir el archivo \n");
25             estado_salida = 1;
26         }
27         else {
28             do {
29                 byte_actual = fgetc(archivo);
30
31                 if (byte_actual != EOF) {
32                     num_aleatorio = rand() % 256;
33
34                     if (byte_actual < num_aleatorio) {
35                         printf("%d-%d\n", num_aleatorio, num_aleatorio - byte_actual);
36                     }
37
38                     if (num_aleatorio < byte_actual) {
39                         printf("%d+%d\n", byte_actual - num_aleatorio, num_aleatorio);
40                     }
41
42                     if (byte_actual == 0) {
43                         printf("|NULL|\n");
44                     }
45
46                     if (num_aleatorio == byte_actual) {
47                         printf("%d+1\n", byte_actual - 1);
48                     }
49                 }
50             } while (byte_actual != EOF);
51
52             fclose(archivo);
53             estado_salida = 0;
54         }
55     }
56     return estado_salida;
57 }
58
59
60
```

Entonces una vez en-
y compilado con la t

Ahora lo probaremos

Ahora lo probaremos para comprobar que funcione igual a lo que hemos planteado.

```
~/Escritorio
> cd gh
~/Escritorio/gh
> wine desconocido.exe juan.txt
0074:fixme:keyboard:klid_from_xkb_layout Unknown variant "mac"
0074:fixme:keyboard:klid_from_xkb_layout Unknown variant "mac-iso"
MESA-EGGL: warning: pci id for fd 33: 10de:2504, driver (null)
pci id for fd 34: 10de:2504, driver (null)
pci id for fd 35: 10de:2504, driver (null)
MESA-EGGL: warning: egl: failed to create dri2 screen
MESA-EGGL: warning: pci id for fd 33: 10de:2504, driver (null)
pci id for fd 34: 10de:2504, driver (null)
pci id for fd 35: 10de:2504, driver (null)
MESA-EGGL: warning: egl: failed to create dri2 screen
MESA-EGGL: warning: pci id for fd 33: 10de:2504, driver (null)
0034:fixme:keyboard:klid_from_xkb_layout Unknown variant "mac"
0034:fixme:keyboard:klid_from_xkb_layout Unknown variant "mac-iso"
59+49
191-94
210-178
152-34
183-86
183-84
144-47
167-135
240-132
86+25
133-25
190-93
9+1
~/Escritorio/gh 6s
DMH-SO
```

```
~/Descargas/tcc
> wine reconstruido.exe juan.txt
62+46
115-18
215-183
195-77
166-69
44+55
21+76
98-66
94+14
119-8
6+102
202-105
248-238
~/Descargas/tcc
> 
```

Bien, ahora podemos demostrarlo comparando las sumas.

Original	Reconstruido
59 +49 = 108	62+46 = 108
191 – 94 = 97	115 – 18 = 97
210 – 178 = 32	215 – 183 = 32
152 – 34 = 118	195 -77 = 118
183 – 84 = 99	44 +55 = 99
144 -47 = 97	21 +76 =97
167 -135 = 32	98 -66 = 32
240 -132 = 208	94 +14 = 208
86 +25 = 111	119 – 8 = 111
133 – 25 = 108	6 + 102 = 108
190 – 93 = 97	202 -105 = 97
9 +1 = 10	248 – 238 = 10

La reconstrucción y análisis del ejecutable se pudo completar de manera satisfactoria. El código fuente en C desarrollado a partir de la descompilación fue validado con la tabla de comparación de arriba, demostrando una correspondencia funcional exacta al replicar el comportamiento, lógica y formato de salida del ejecutable original bajo las mismas condiciones de entrada, esto no hubiera sido posible sin el uso de las herramientas como ghidra, tcc y conocimientos básicos de los números hexadecimales.

Bibliografía:

[https://www-varonis-com.translate.goog/blog/how-to-use-ghidra?
_x_tr_sl=en&_x_tr_tl=es&_x_tr_hl=es&_x_tr_pto=tc](https://www-varonis-com.translate.goog/blog/how-to-use-ghidra?_x_tr_sl=en&_x_tr_tl=es&_x_tr_hl=es&_x_tr_pto=tc)